Algorithmes d’apprentissage non supervisé

02-03

Locality-Sensitive Hashing

@ 2026 MKL.AI - Tous droits réservés.

Au programme

- Plus proches voisins: considérations pratiques
- Algorithme LSH
- Implémentation
- Nombre de bins
- Qualité de la recherche
- LSH et dimensionnalité
- Atelier
- Lectures et références

@ 2026 MKL.AI - Tous droits réservés.

NNPlus proches voisins: considérations pratiques

Plus proches voisins: considérations pratiques

- La recherche d’un plus proche voisin par **force brute** ou par l’algorithme **KD-trees** (non couvert par le cours) ne résout pas certaines problématiques:
  - Implémentation efficace difficile (KD-trees)
  - Fléau de la dimension (force brute/KD-trees)

- Pour un grand nombre d’application, il n’est pas nécessaire de trouver le plus proche voisin de manière exacte
  - Sur plusieurs millions d’articles, a t-on nécessairement besoin du plus proche ou plus simplement d’un article similaire ?
  - Quelle confiance accorder à la mesure de similarité ?

@ 2026 MKL.AI - Tous droits réservés.

Plus proches voisins: considérations pratiques

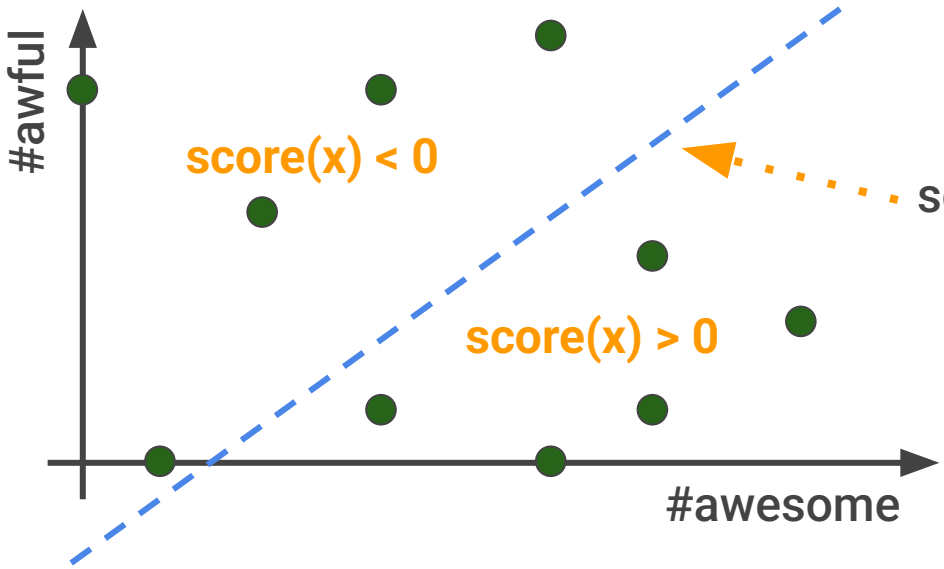
- La recherche d’un plus proche voisin par **force brute** ou par l’algorithme **KD-trees** (non couvert par le cours) ne résout pas certaines problématiques:
  - Implémentation efficace difficile (KD-trees)
  - Fléau de la dimension (force brute/KD-trees)
- Pour un grand nombre d’application, il n’est pas nécessaire de trouver le plus proche voisin de manière exacte
  - Sur plusieurs millions d’articles, a t-on nécessairement besoin du plus proche ou plus simplement d’un article similaire ?
  - Quelle confiance accorder à la mesure de similarité ?

Besoin d’une méthode retournant un résultat **approximatif** ayant une forte probabilité d’être proche du document “requête”

@ 2026 MKL.AI - Tous droits réservés.

Algorithme LSH

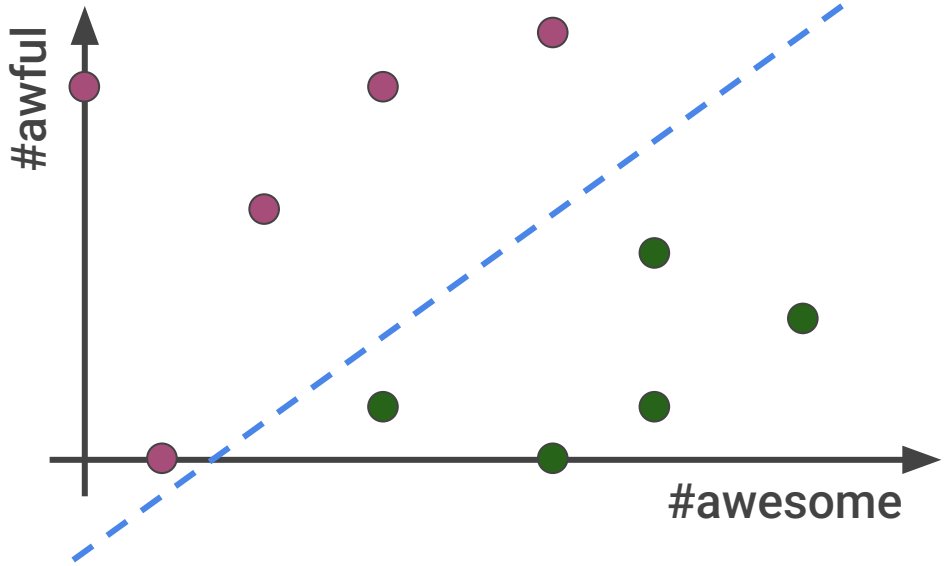
“Binning” simple des données (1/3)



score(x) = 1.0 #awesome – 1.5 #awful

@ 2026 MKL.AI - Tous droits réservés.

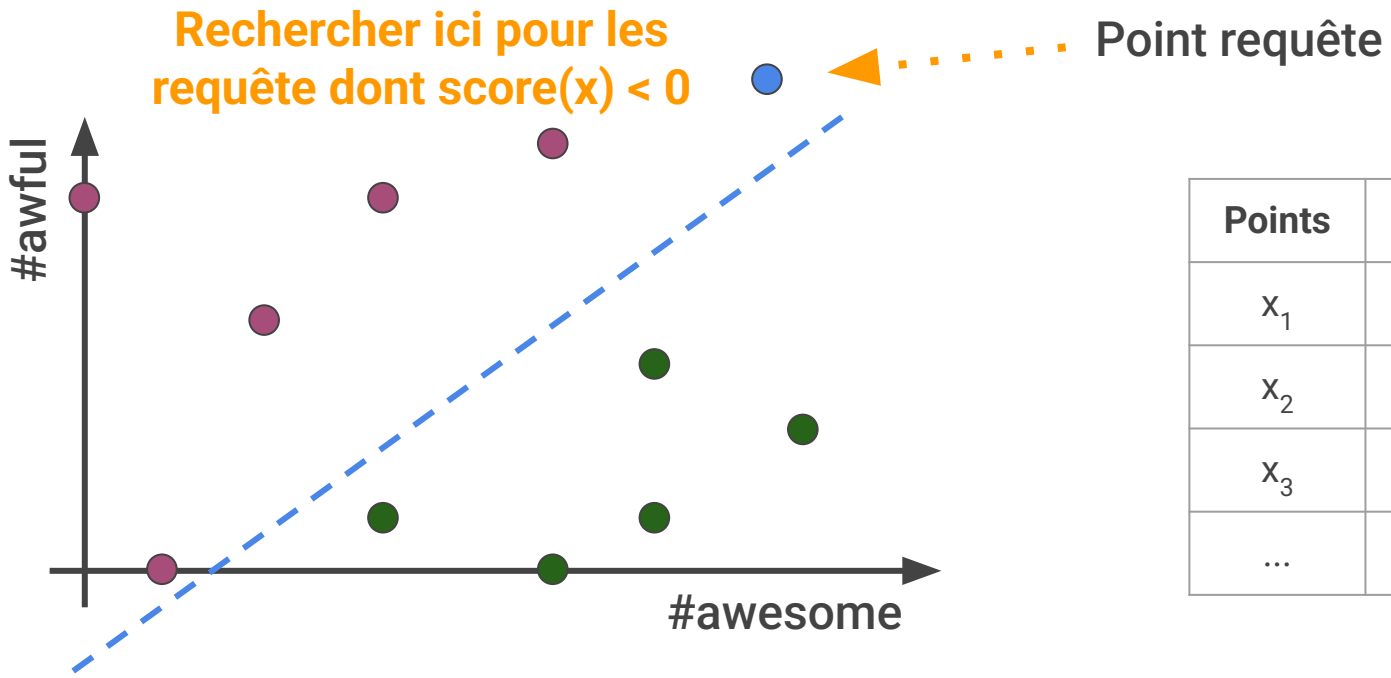
“Binning” simple des données (2/3)



Points	Score
x_1	-
x_2	-
x_3	+
...	...

@ 2026 MKL.AI - Tous droits réservés.

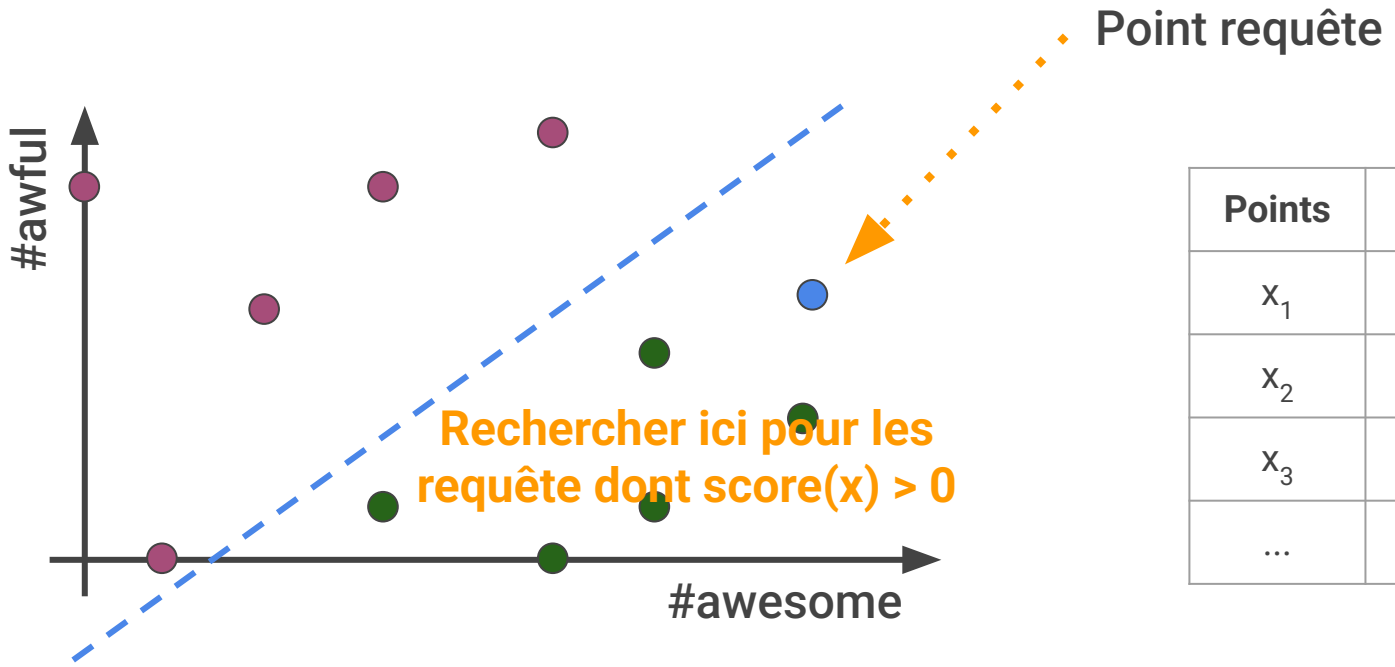
“Binning” simple des données (2/3)



Points	Score	Bin
x_1	-	0
x_2	-	0
x_3	+	1
...	...	...

@ 2026 MKL.AI - Tous droits réservés.

“Binning” simple des données (2/3)



Points	Score	Bin
x_1	-	0
x_2	-	0
x_3	+	1
...	...	...

@ 2026 MKL.AI - Tous droits réservés.

“Binning” simple des données (3/3)

Index du bin

Bin	0	1
Liste des indices des points	{1, 2, 3, 7, …}	{3, 5, 6, 8, …}

Points	Score	Bin
x_1	-	0
x_2	-	0
x_3	+	1
…	…	…

@ 2026 MKL.AI - Tous droits réservés.

“Binning” simple des données (3/3)

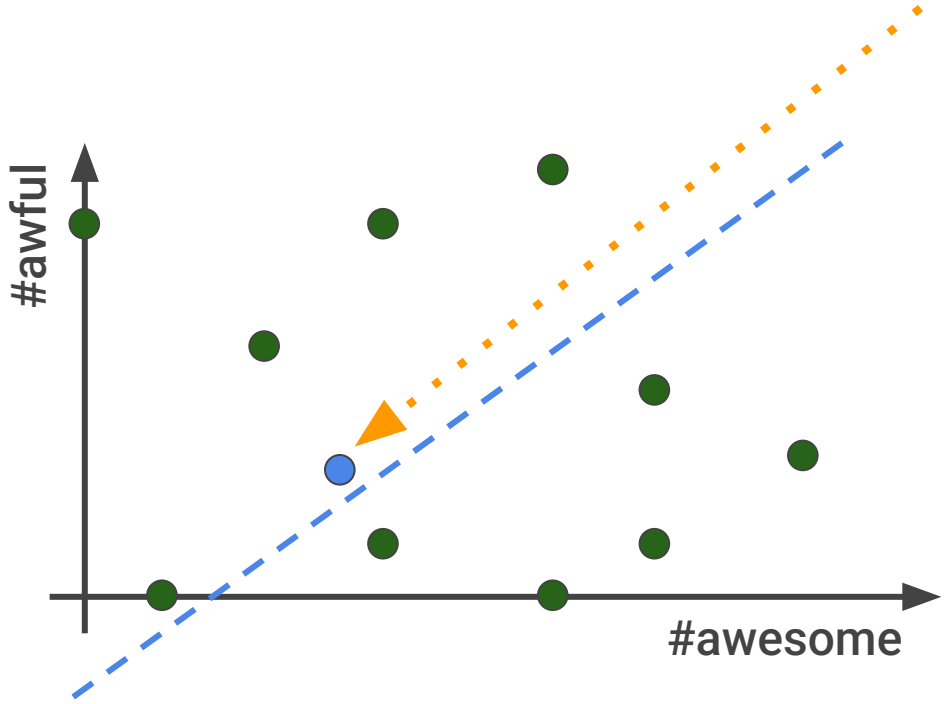
Index du bin

Bin	0	1
Liste des indices des points	{1, 2, 7, …}	{3, 5, 6, 8, …}

Points	Score	Bin
x_1	-	0
x_2	-	0
x_3	+	1
…	…	…

@ 2026 MKL.AI - Tous droits réservés.

Approximation NN



Avec cette partition, est-on capable de trouver “le” plus proche voisin de ce point requête? Non !

@ 2026 MKL.AI - Tous droits réservés.

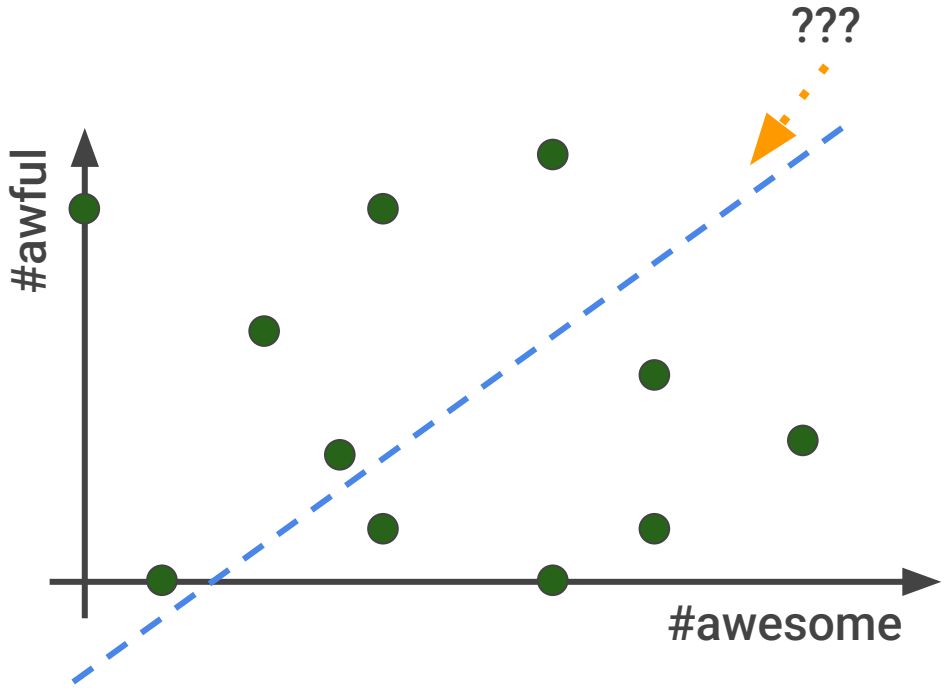
3 problématiques persistent ...

- Comment trouver une **ligne de partition** ?
- Qualité de la solution (des points proches peuvent être placés dans des bins différents
- Coût de calculs: les bins contiennent encore trop de points, espace de recherche trop grand pour l’algorithme des plus proches voisins

@ 2026 MKL.AI - Tous droits réservés.

Implémentation

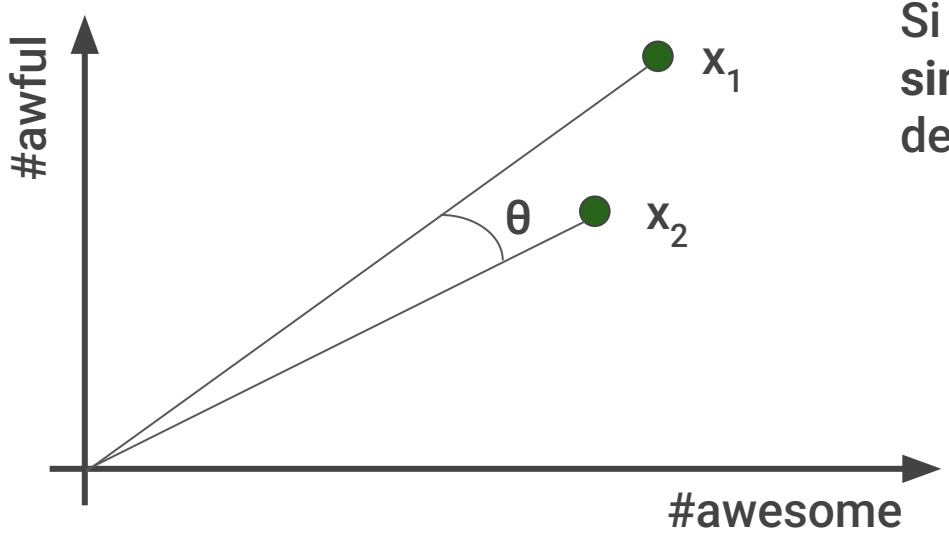
Comment trouver une ligne de partition ?



Au hasard !

@ 2026 MKL.AI - Tous droits réservés.

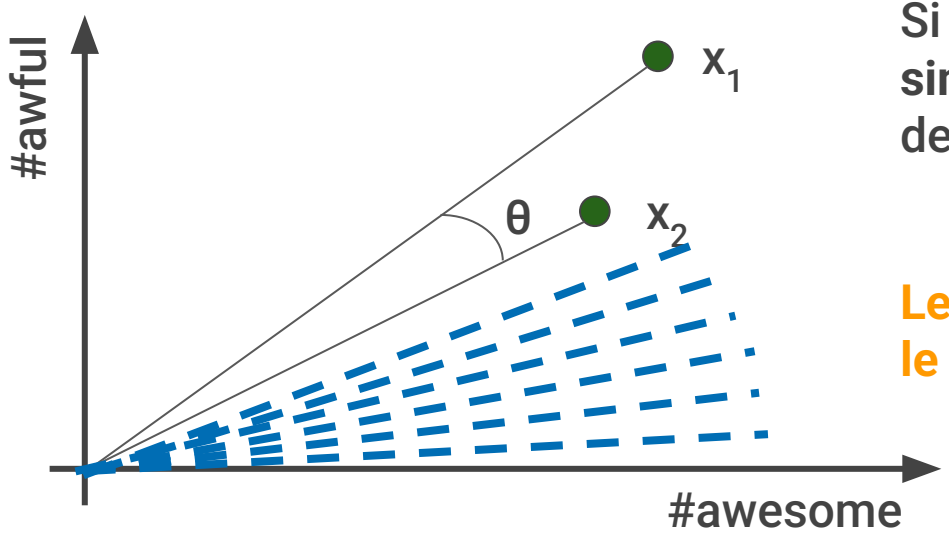
Comment trouver une ligne de partition ?



Si x_1 et x_2 sont proches (d'un point de vue **similarité cosinus**), alors ceux-ci devraient se retrouver dans le même bin

@ 2026 MKL.AI - Tous droits réservés.

Comment trouver une ligne de partition ?

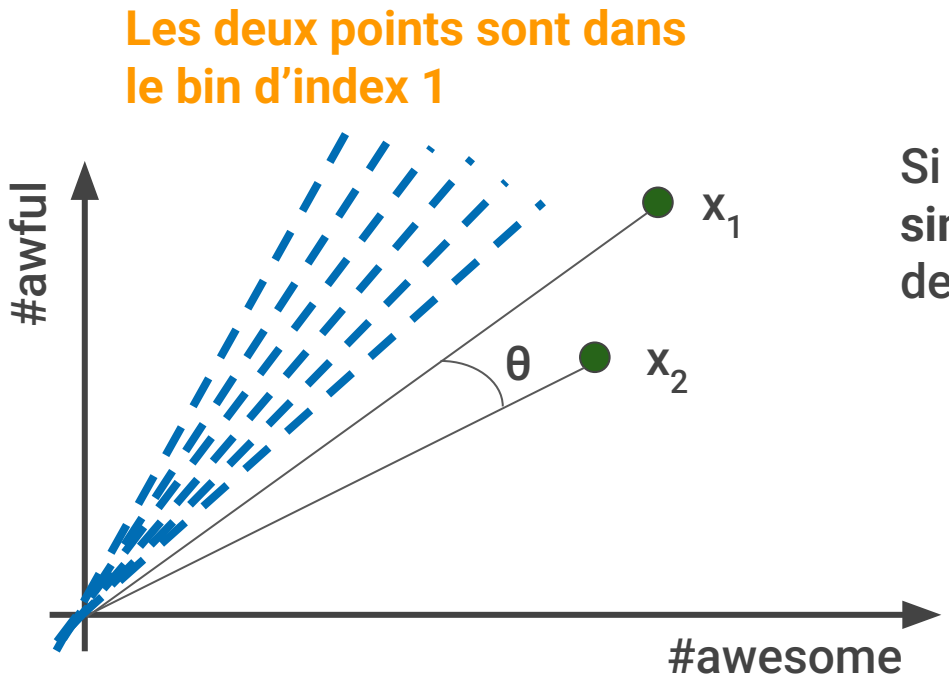


Si x_1 et x_2 sont proches (d'un point de vue **similarité cosinus**), alors ceux-ci devraient se retrouver dans le même bin

Les deux points sont dans le bin d'index 0

@ 2026 MKL.AI - Tous droits réservés.

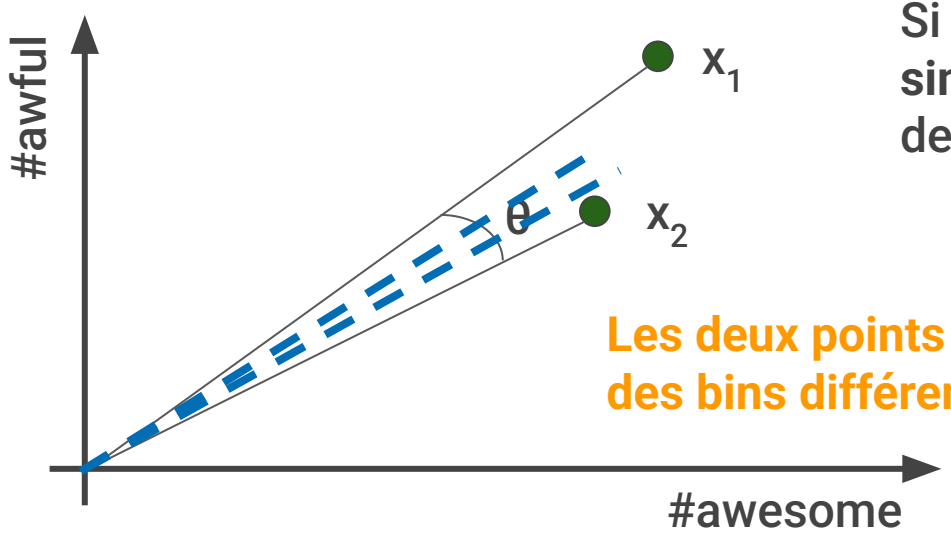
Comment trouver une ligne de partition ?



Si x_1 et x_2 sont proches (d'un point de vue **similarité cosinus**), alors ceux-ci devraient se retrouver dans le même bin

@ 2026 MKL.AI - Tous droits réservés.

Comment trouver une ligne de partition ?

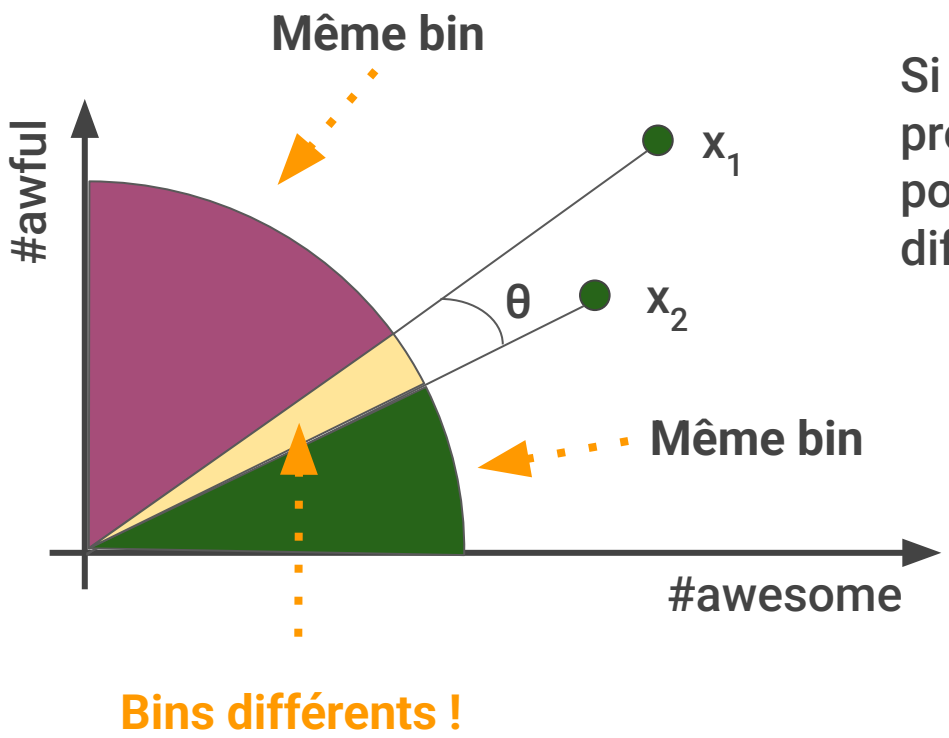


Si x_1 et x_2 sont proches (d'un point de vue **similarité cosinus**), alors ceux-ci devraient se retrouver dans le même bin

Les deux points sont dans des bins différents

@ 2026 MKL.AI - Tous droits réservés.

Comment trouver une ligne de partition ?



Si θ est suffisamment petit (x_1 et x_2 sont proches), alors la probabilité que les points soient placés dans des bins différents est faible

@ 2026 MKL.AI - Tous droits réservés.

3 problématiques persistent ...

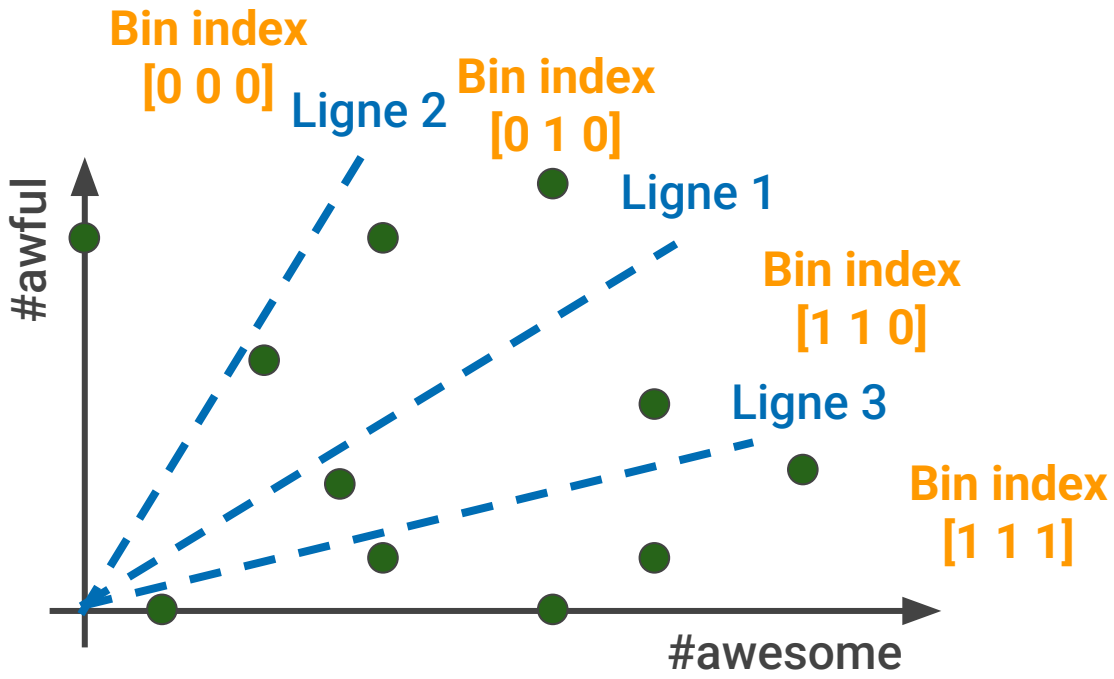
- Comment trouver une **ligne de partition** ? ✔
- Qualité de la solution (des points proches peuvent être placés dans des bins différents ✔
- **Coût de calculs: les bins contiennent encore trop de points, espace de recherche trop grand pour NN**

Bin	0	1
Liste des indices des points	{1, 2, 3, 7, …}	{3, 5, 6, 8, …}

@ 2026 MKL.AI - Tous droits réservés.

Nombre de bins

Réduction du coût de recherche avec plusieurs bins



@ 2026 MKL.AI - Tous droits réservés.

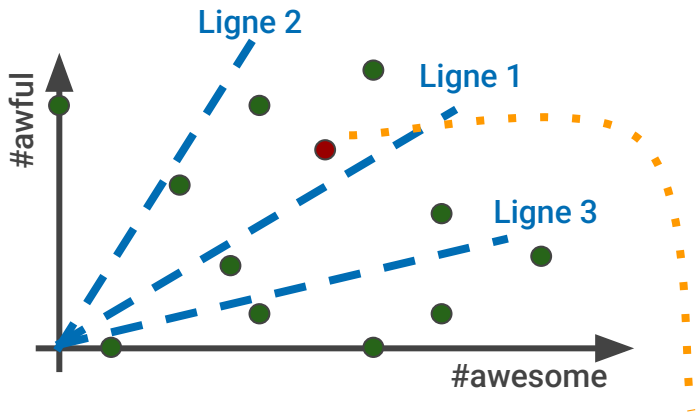
Réduction du coût de recherche avec plusieurs bins

Points	Score 1	Index bin 1	Score 2	Index bin 2	Score 3	Index bin 3
x_1	-	0	-	0	-	0
x_2	-	0	-	0	-	0
x_3	+	1	+	1	+	1
…	…	…	…	…	…	…

Bin	[0 0 0] 0	[0 0 1] 1	[0 1 0] 2	[0 1 1] 3	[1 0 0] 4	[1 0 1] 5	[1 1 0] 6	[1 1 1] 7
Liste des indices des points	{1, 2}	--	{4, 8, 11}	--	--	--	{7, 9, 10}	{3, 5, 6}

@ 2026 MKL.AI - Tous droits réservés.

Réduction du coût de recherche avec plusieurs bins



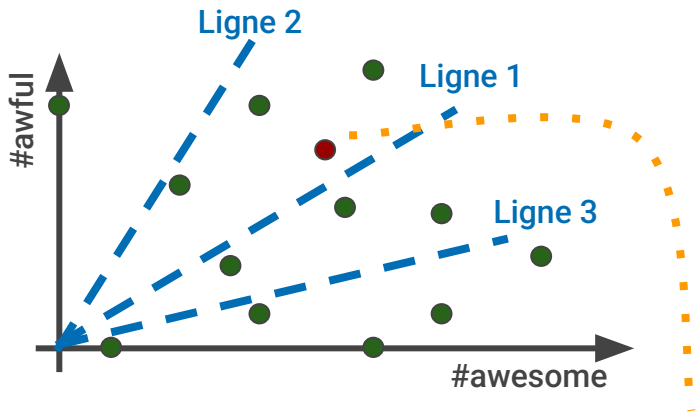
Recherche des plus proches voisins parmi les éléments du bin d'indice 2

Bin	[0 0 0] 0	[0 0 1] 1	[0 1 0] 2	[0 1 1] 3	[1 0 0] 4	[1 0 1] 5	[1 1 0] 6	[1 1 1] 7
Liste des indices des points	{1, 2}	--	{4, 8, 11}	--	--	--	{7, 9, 10}	{3, 5, 6}

@ 2026 MKL.AI - Tous droits réservés.

Qualité de la recherche

Qualité de la recherche



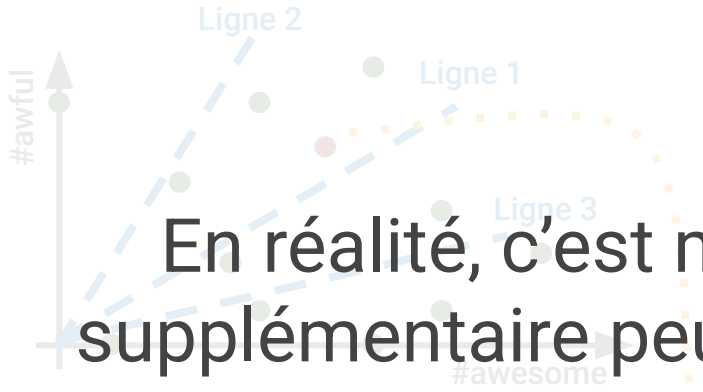
Le point requête est ici, mais le NN est-il nécessairement dans le bin d'index 2 ?

Pas forcément ...

Bin	[0 0 0] 0	[0 0 1] 1	[0 1 0] 2	[0 1 1] 3	[1 0 0] 4	[1 0 1] 5	[1 1 0] 6	[1 1 1] 7
Liste des indices des points	{1, 2}	--	{4, 8, 11}	--	--	--	{7, 9, 10}	{3, 5, 6}

@ 2026 MKL.AI - Tous droits réservés.

Qualité de la recherche



Le point requête est ici, mais le NN est-il nécessairement dans ce bin ?

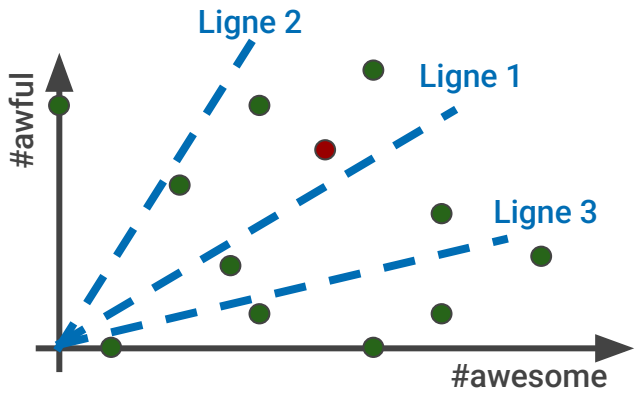
En réalité, c’est même pire car chaque ligne supplémentaire peut séparer des points proches

Compromis vitesse - précision

Bin	[0 0 0] 0	[0 0 1] 1	[0 1 0] 2	[0 1 1] 3	[1 0 0] 4	[1 0 1] 5	[1 1 0] 6	[1 1 1] 7
Liste des indices des points	{1, 2}	--	{4, 8, 11}	--	--	--	{7, 9, 10}	{3, 5, 6}

@ 2026 MKL.AI - Tous droits réservés.

Qualité de la recherche

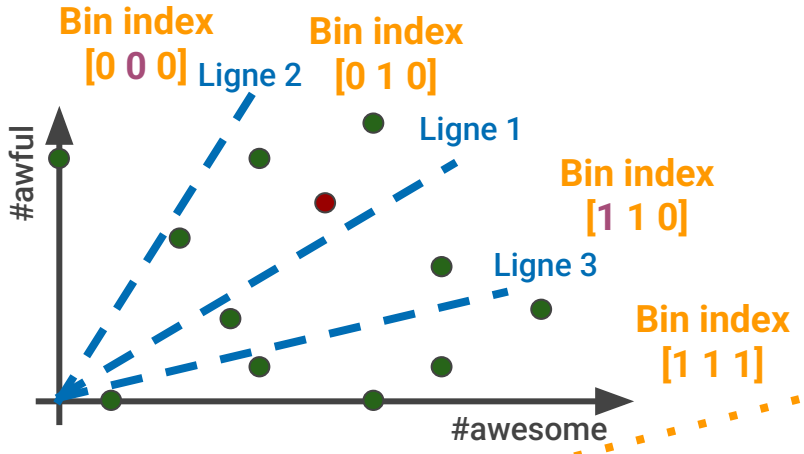


Dans ce cas, il peut être intéressant de rechercher dans **les bins les plus proches**. Par quelle opération peut-on les trouver?

Bin	[0 0 0] 0	[0 0 1] 1	[0 1 0] 2	[0 1 1] 3	[1 0 0] 4	[1 0 1] 5	[1 1 0] 6	[1 1 1] 7
Liste des indices des points	{1, 2}	--	{4, 8, 11}	--	--	--	{7, 9, 10}	{3, 5, 6}

@ 2026 MKL.AI - Tous droits réservés.

Qualité de la recherche

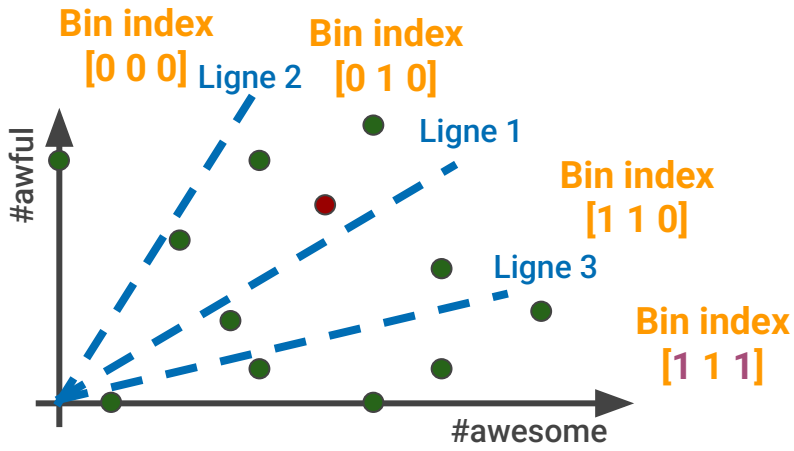


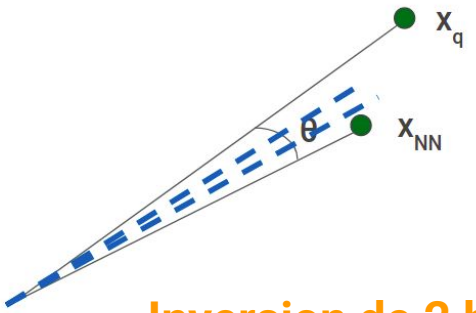
Inversion de 1 bit

Bin	[0 0 0] 0	[0 0 1] 1	[0 1 0] 2	[0 1 1] 3	[1 0 0] 4	[1 0 1] 5	[1 1 0] 6	[1 1 1] 7
Liste des indices des points	{1, 2}	--	{4, 8, 11}	--	--	--	{7, 9, 10}	{3, 5, 6}

@ 2026 MKL.AI - Tous droits réservés.

Qualité de la recherche





Inversion de 2 bits

Bin	[0 0 0] 0	[0 0 1] 1	[0 1 0] 2	[0 1 1] 3	[1 0 0] 4	[1 0 1] 5	[1 1 0] 6	[1 1 1] 7
Liste des indices des points	{1, 2}	--	{4, 8, 11}	--	--	--	{7, 9, 10}	{3, 5, 6}

@ 2026 MKL.AI - Tous droits réservés.

Qualité de la recherche

- La qualité du NN trouvé ne peut qu’augmenter en recherchant dans plus de bins
- Algorithme: **continuer la recherche du NN tant que le budget de calcul n’est dépassé ou que la qualité du meilleur NN trouvé soit suffisamment satisfaisante**

@ 2026 MKL.AI - Tous droits réservés.

LSH - Résumé

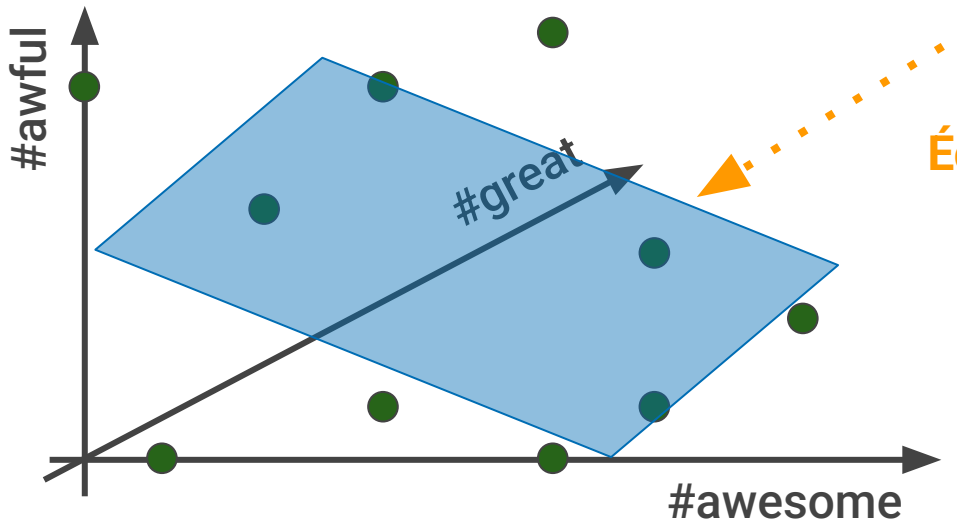
- Échantillonner **h** lignes de partition ?
- Calculer le score pour chaque point et convertir en index binaire
- Utiliser un vecteur binaire de **h** bits par point comme index de bin
- Créer la table de hachage
- Pour chaque point requête $\mathbf{x}_q$, rechercher bin($\mathbf{x}_q$), puis les bins voisins jusqu'à un critère d'arrêt prédéfini

@ 2026 MKL.AI - Tous droits réservés.

LSH et dimensionnalité

LSH et dimensionnalité

$$\text{score}(x) = v_1 \ \#\text{awesome} + v_2 \ \#\text{awful} + v_2 \ \#\text{great}$$



Échantillonner des plans aléatoires

@ 2026 MKL.AI - Tous droits réservés.

Atelier

https://github.com/mswawola-cegep/420-a58-sf-gr-12022-hiver-2026.git

02-03

Lectures et références

Références

[1] Machine Learning: Clustering and Retrieval - Emily Fox & Carlos Guestrin - University of Washington

[2] Jure Leskovec, Anand Rajaraman, and Jeffrey D. Ullman, “Mining of Massive Datasets”

@ 2026 MKL.AI - Tous droits réservés.